

Count Based RNA-seq analysis

Kasper D. Hansen

- [Dependencies](#)
- [Overview](#)
- [Other Resources](#)
- [RNA-seq count data](#)
- [Statistical issues](#)
- [The Data](#)
- [edgeR](#)
- [DESeq2](#)
- [Comments](#)
- [SessionInfo](#)

Dependencies

This document has the following dependencies:

```
library(DESeq2)
library(edgeR)
```

Use the following commands to install these packages in R.

```
source("http://www.bioconductor.org/biocLite.R")
biocLite(c("DESeq2", "edgeR"))
```

Overview

RNA seq data is often analyzed by creating a count matrix of gene counts per sample. This matrix is analyzed using count-based models, often built on the negative binomial distribution. Popular packages for this includes [edgeR](#) and [DESeq](#) / [DESeq2](#).

This type of analysis discards part of the information in the RNA sequencing reads, but we have a good understanding of how to analyze this type of data.

Other Resources

- The vignette from the [edgeR webpage](#).
- The vignette from the [DESeq2 webpage](#).
- The [RNA-seq workflow](#).

RNA-seq count data

One simple way of analyzing RNA sequencing data is to make it look like microarray data. This is done by counting how many reads in each sample overlaps a gene. There are many ways to do this. It obviously depends on the annotation used, but also on how it is decided that a read overlaps a region. Of specific concern is which genomic regions are part of a gene with multiple transcripts.

There are no consensus on this process and the different choices one make is known to affect the outcome.

Tools for doing gene counting includes

- by using `featureCounts()` from the [Rsubread](#) package.
- the [HTSeq](#) package (this is a python package, not a Bioconductor package).
- by using `summarizeoverlaps()` from the [GenomicAlignments](#) package.

and there are other alternatives. Many people seem to write their own counting pipeline.

Reducing RNA sequencing data to a single integer per gene is obvious a simplification. Indeed it ignores some of the main reasons for doing RNA sequencing, including assessing alternative splicing. On the other hand, we understand the statistical properties of this procedure well, and it delivers a basic insight into something that most researcher wants to know. Finally, this approach requires the different genomic regions to be known beforehand.

Statistical issues

In RNA-seq data analysis we often see that many genes (up to 50%) have little or no expression. It is common to pre-filter (remove) these genes prior to analysis. In general genomics filtering might be beneficial to your analysis, but this discussion is outside the scope of this document.

Note: The analysis presented below is **extremely** superficial. Consider this a very quick introduction to the workflow of these two packages.

The Data

We will be using the [airway](#) dataset which contains RNA-seq data in the form of a SummarizedExperiment. Lets load the data and have a look

```
library(airway)
data(airway)
airway
```

```
## class: SummarizedExperiment
## dim: 64102 8
## exptData(1): ''
## assays(1): counts
## rownames(64102): ENSG000000000003 ENSG000000000005 ... LRG_98 LRG_99
## rowRanges metadata column names(0):
## colnames(8): SRR1039508 SRR1039509 ... SRR1039520 SRR1039521
## colData names(9): SampleName cell ... Sample BioSample
```

```
assay(airway, "counts")[1:3, 1:3]
```

```
##                SRR1039508 SRR1039509 SRR1039512
## ENSG000000000003         679         448         873
## ENSG000000000005          0          0          0
## ENSG000000000419         467         515         621
```

```
airway$dex
```

```
## [1] untrt trt   untrt trt   untrt trt   untrt trt
## Levels: trt untrt
```

The main variable of interest is dex which takes on levels trt (treated) and untrt (untreated). The first level will be the reference level for this factor, so we use `relevel()` to set the untrt level as reference; this is much easier to interpret.

```
airway$dex <- relevel(airway$dex, "untrt")
airway$dex
```

```
## [1] untrt trt    untrt trt    untrt trt    untrt trt
## Levels: untrt trt
```

There is rich information about which gene model was used for each gene:

```
granges(airway)
```

```
## GRangesList object of length 64102:
## $ENSG000000000003
## GRanges object with 17 ranges and 2 metadata columns:
##      seqnames          ranges strand | exon_id      exon_
##      <Rle>             <IRanges> <Rle> | <integer>    <charac
## [1]      X [99883667, 99884983]     - |    667145 ENSE0000145
## [2]      X [99885756, 99885863]     - |    667146 ENSE0000086
## [3]      X [99887482, 99887565]     - |    667147 ENSE0000040
## [4]      X [99887538, 99887565]     - |    667148 ENSE0000184
## [5]      X [99888402, 99888536]     - |    667149 ENSE0000355
## ...      ...                ...    ...
## [13]     X [99890555, 99890743]     - |    667156 ENSE0000351
## [14]     X [99891188, 99891686]     - |    667158 ENSE0000188
## [15]     X [99891605, 99891803]     - |    667159 ENSE0000185
## [16]     X [99891790, 99892101]     - |    667160 ENSE0000186
## [17]     X [99894942, 99894988]     - |    667161 ENSE0000182
##
## ...
## <64101 more elements>
## -----
## seqinfo: 722 sequences (1 circular) from an unspecified genome
```

edgeR

The *edgeR* is very similar in terms of data structures and functionality to the *limma*. Whereas *limma* allows us to operate directly on ExpressionSets, edgeR does not work directly with SummarizedExperiment. We first need to put our data into an edgeR specific container.

```
library(edgeR)
dge <- DGEList(counts = assay(airway, "counts"),
               group = airway$dex)
dge$samples <- merge(dge$samples,
                    as.data.frame(colData(airway)),
                    by = 0)
dge$genes <- data.frame(name = names(rowRanges(airway)),
                       stringsAsFactors = FALSE)
```

This object has something called the group which is the basic experimental group for each sample. It also has \$samples (the pheno data) which - weirdly - cannot be set when you create the DGEList object, so we set it afterwards. The \$genes is a data.frame so we cannot include the rich gene model information we had in the SummarizedExperiment.

Having set up the input object, we now proceed as follows.

First we estimate the normalization factors or effective library sizes

```
dge <- calcNormFactors(dge)
```

Next we setup the design matrix and estimate the dispersion (variance). There are multiple ways to do this, and the weird two-step procedure is necessary.

```
design <- model.matrix(~dge$samples$group)
dge <- estimateGLMCommonDisp(dge, design)
dge <- estimateGLMTagwiseDisp(dge, design)
```

Now we do a glmFit(), similar to limma

```
fit <- glmFit(dge, design)
```

Now it is time to do a test and extract the top hits

```
lrt <- glmLRT(fit, coef = 2)
topTags(lrt)
```

```
## Coefficient: dge$samples$grouptrt
##          name      logFC  logCPM      LR      PValue
## 9658  ENSG00000152583  4.584952  5.536240  286.3965  3.032129e-64
## 14922 ENSG00000179593 10.100345  1.658949  180.1177  4.568028e-41
## 3751  ENSG00000109906  7.128577  4.162779  170.6604  5.307950e-39
## 44236 ENSG00000250978  6.166269  1.398266  168.8572  1.314558e-38
## 14827 ENSG00000179094  3.167788  5.177198  161.6348  4.971441e-37
## 17245 ENSG00000189221  3.289112  6.769209  138.9111  4.606056e-32
## 5054  ENSG00000120129  2.932939  7.310715  137.0461  1.178199e-31
## 2529  ENSG00000101347  3.842550  9.207488  131.4672  1.956855e-30
## 2071  ENSG00000096060  3.921841  6.898829  123.3973  1.141438e-28
## 14737 ENSG00000178695 -2.515219  6.959467  122.9711  1.414932e-28
##          FDR
## 9658  1.943655e-59
## 14922 1.464099e-36
## 3751  1.134167e-34
## 44236 2.106644e-34
## 14827 6.373586e-33
## 17245 4.920957e-28
## 5054  1.078927e-27
## 2529  1.567979e-26
## 2071  8.129829e-25
## 14737 9.069997e-25
```

DESeq2

Like *edgeR*, DESeq2 requires us to put the data into a package-specific container (a *DESeqDataSet*). But unlike *edgeR*, it is pretty easy.

```
library(DESeq2)
dds <- DESeqDataSet(airway, design = ~ dex)
```

Note that the design of the experiment is stored inside the object. The last variable (in case multiple variables are list) will be the variable of interest which is report in the different results outputs.

Fitting the model is simple

```
dds <- DESeq(dds)
```

and then all we need to do is get the results. Note that the results are not ordered, so we do that.

```
res <- results(dds)
res <- res[order(res$padj),]
res
```

```
## log2 fold change (MAP): dex trt vs untrt
## wald test p-value: dex trt vs untrt
## DataFrame with 64102 rows and 6 columns
##           baseMean log2FoldChange      lfcSE      stat
##           <numeric>      <numeric> <numeric> <numeric>
## ENSG00000152583    997.4398      4.285847 0.19605831 21.86006
## ENSG00000148175  11193.7188      1.434388 0.08411178 17.05336
## ENSG00000179094    776.5967      2.984244 0.18864120 15.81968
## ENSG00000109906    385.0710      5.137007 0.33077733 15.53010
## ENSG00000134686   2737.9820      1.368176 0.09059205 15.10261
## ...           ...           ...           ...           ...
## LRG_94              0              NA          NA          NA
## LRG_96              0              NA          NA          NA
## LRG_97              0              NA          NA          NA
## LRG_98              0              NA          NA          NA
## LRG_99              0              NA          NA          NA
##           pvalue      padj
##           <numeric> <numeric>
## ENSG00000152583 6.235873e-106 1.089469e-101
## ENSG00000148175 3.300017e-65 2.882730e-61
## ENSG00000179094 2.276377e-56 1.325686e-52
## ENSG00000109906 2.170243e-54 9.479080e-51
## ENSG00000134686 1.556576e-51 5.438987e-48
## ...           ...           ...
## LRG_94              NA          NA
## LRG_96              NA          NA
## LRG_97              NA          NA
## LRG_98              NA          NA
## LRG_99              NA          NA
```

Comments

We see that amongst the top 5 genes, 3 are shared between edgeR and DESeq2, with some small variation in the estimated fold-change. The two methods are both being continually developed (and probably bench-marked against each other by the authors). At any given time it is difficult to decide which one to prefer.

SessionInfo

```
## R version 3.2.1 (2015-06-18)
## Platform: x86_64-apple-darwin13.4.0 (64-bit)
## Running under: OS X 10.10.5 (Yosemite)
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## attached base packages:
## [1] parallel stats4 methods stats graphics grDevices utils
## [8] datasets base
##
## other attached packages:
## [1] airway_0.102.0 edgeR_3.10.2
## [3] limma_3.24.15 DESeq2_1.8.1
## [5] RcppArmadillo_0.5.500.2.0 Rcpp_0.12.0
## [7] GenomicRanges_1.20.6 GenomeInfoDb_1.4.2
## [9] IRanges_2.2.7 S4Vectors_0.6.5
## [11] BiocGenerics_0.14.0 BiocStyle_1.6.0
## [13] rmarkdown_0.8
##
## loaded via a namespace (and not attached):
## [1] RColorBrewer_1.1-2 formatR_1.2 futile.logger_1.4.1
## [4] plyr_1.8.3 xVector_0.8.0 futile.options_1.0.0
## [7] tools_3.2.1 rpart_4.1-10 digest_0.6.8
## [10] RSQLite_1.0.0 annotate_1.46.1 evaluate_0.7.2
## [13] gtable_0.1.2 lattice_0.20-33 DBI_0.3.1
## [16] yaml_2.1.13 proto_0.3-10 gridExtra_2.0.0
## [19] genefilter_1.50.0 cluster_2.0.3 stringr_1.0.0
## [22] knitr_1.11 locfit_1.5-9.1 nnet_7.3-10
## [25] grid_3.2.1 Biobase_2.28.0 AnnotationDbi_1.30.1
## [28] XML_3.98-1.3 survival_2.38-3 BiocParallel_1.2.20
## [31] foreign_0.8-65 latticeExtra_0.6-26 Formula_1.2-1
## [34] geneplotter_1.46.0 ggplot2_1.0.1 reshape2_1.4.1
## [37] lambda.r_1.1.7 magrittr_1.5 scales_0.3.0
## [40] Hmisc_3.16-0 htmltools_0.2.6 MASS_7.3-43
## [43] splines_3.2.1 xtable_1.7-4 colorspace_1.2-6
## [46] stringi_0.5-5 acepack_1.3-3.3 munsell_0.4.2
```